

Reviewer Pack — Minimal Geometric Fluctuation Bound on Communication Complex...

Entry 721875b3818c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 11:11:29 UTC

Minimal Geometric Fluctuation Bound on Communication Complexity Rank Variance

Entry ID: 721875b3818c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 11:11:29 UTC

1. Conjecture

Field A (mathematical branch): Geometric Measure Theory (Fractal Dimension) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the geometric fluctuation (G_f) of its associated hypercube partitioned by f is linearly correlated with the variance of its matrix representation ($\text{rank}(f)$), such that $G_f = O(\text{rank}(f)^2)$.

Rationale (proposer's reasoning):

Geometric fluctuation measures the irregularity in the distribution of points, which could reflect the complexity of the Boolean function. Communication complexity rank is a measure of the difficulty of computing the function with minimal communication. This conjecture suggests that higher geometric fluctuation corresponds to higher communication complexity, potentially uncovering new insights into the interplay between these two fields.

Taxonomy category: Fractal Complexity vs Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8025e4e6829985c8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the correlation coefficient between G_f and $\text{rank}(\text{rank}(f))$ exceeds 0.9 across at least 100 independent seeds, then support is provided; otherwise, if any seed has a correlation coefficient below 0.5 or any single G_f value exceeds $20 * (\text{rank}(f))^2$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric fluctuation" AND "communication complexity" AND "matrix rank" - "fractal dimension" IN title AND "communication complexity" IN title - "hypercube partitioning" AND "Boolean function" AND "variance of matrix representation"

Top relevant hits considered: - [http://arxiv.org/abs/1810.12657v1] The effect of a country's name in the title of a publication on its visibility and citability - [http://arxiv.org/abs/1501.04911v2] Fractal dimension and turbulence in Giant HII Regions - [http://arxiv.org/abs/cs/0207003v1] Analysis of Titles and Readers For Title Generation Centered on the Readers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def matrix_representation(f, n):
        S = list(range(2**n))
        M = [[f(S[i] ^ S[j]) for j in range(n)] for i in range(n)]
        return M

    def geometric_fluctuation(M):
        n = len(M)
        mean = sum(sum(row) for row in M) / (n * n)
        variance = sum((M[i][j] - mean)**2 for i in range(n) for j in range(n)) / (n * n)
        return math.sqrt(variance)

    def rank(matrix):
        n = len(matrix)
        A = [row[:] for row in matrix]
        pivot_row = 0
```

```

for col in range(n):
    if all(A[row][col] == 0 for row in range(pivot_row, n)):
        continue
    max_row = pivot_row
    for i in range(pivot_row + 1, n):
        if abs(A[i][col]) > abs(A[max_row][col]):
            max_row = i
    A[pivot_row], A[max_row] = A[max_row], A[pivot_row]
    for row in range(n):
        if row != pivot_row:
            factor = -A[row][col] / A[pivot_row][col]
            for j in range(col, n):
                A[row][j] += factor * A[pivot_row][j]
    pivot_row += 1
return pivot_row

def correlation(x, y):
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))) / len(x)
    std_x = math.sqrt(sum((x[i] - mean_x)**2 for i in range(len(x))) / len(x))
    std_y = math.sqrt(sum((y[i] - mean_y)**2 for i in range(len(y))) / len(y))
    return cov / (std_x * std_y)

n_values = [5, 10, 15, 20, 30, 40]
metric_values = []
instances_tested = 0
n_max = 0

for n in n_values:
    f = generate_boolean_function(n)
    M = matrix_representation(f, n)
    G_f = geometric_fluctuation(M)
    rank_f = rank(M)

    if G_f > 20 * (rank_f ** 2):
        return {
            "metric_name": "geometric_fluctuation",
            "metric_value": G_f,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": f"Geometric fluctuation {G_f} exceeds 20 * (rank(f)^2) = {20 * (rank_f ** 2)}"
        }

    metric_values.append(G_f)
    instances_tested += 1
    n_max = max(n_max, n)

correlation_coefficient = correlation(metric_values, [rank(matrix_representation(generate_boolean_function(n))) for n in n_values])

return {
    "metric_name": "geometric_fluctuation",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
}

```

```

        "conjecture_holds": correlation_coefficient > 0.9,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(result["metric_value"] < 0.5 or result["counterexample"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result["metric_value"] < 0.5 or result["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31756259.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31756259.py", line 70, in run_trial
    M = matrix_representation(f, n)
       ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31756259.py", line 26, in matrix_representation
    M = [[f(S[i] ^ S[j]) for j in range(n)] for i in range(n)]
         ~~~~~^~~~~~
TypeError: 'list' object is not callable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture’s validity. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14366
2	preregistration	ollama_remote	glm4:latest	0	0	9844
3	novelty	ollama_remote	glm4:latest	0	0	8767
4	novelty	ollama_remote	glm4:latest	0	0	9676
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	46358
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14317
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14122
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14474
9	judge	ollama_remote	glm4:latest	0	0	15897

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 147821 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/721875b3818c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/721875b3818c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/721875b3818c.tar.gz (if generated)